

sembling the components on the PCB, you need to interface the board with the PC for data exchange. For that, you have software that communicates with the physical hardware through USB interface using its device driver dynamic link library (DLL). The graphical-user-interface (GUI) software was developed using Microsoft Visual C# 2010. net and NI LabView Version 8.6 (2008).

The hardware developed has the firmware loaded onto the Flash memory of the microcontroller on the DAQ board. To design the GUI program, first you need to know how the firmware in the board and the PC should communicate.

The firmware continuously sends

and receives eight bytes of data to and from the PC. Ports C and D of the microcontroller are connected to CON1 digital I/O pins. The first two bytes from the PC contain data for data direction registers (DDRs), namely, DDRC and DDRD. The next two consecutive bytes, i.e., the third and the fourth bytes, contain the value that is to be written to each port. Each bit in the register DDRx and PORTx values corresponds to a physical pin of the microcontroller's ports C and D. For example, to set the fourth and sixth pins as outputs, you need to write to the DDRx register a value equivalent to the binary value '00101000.' To set the fourth pin 'high' you need

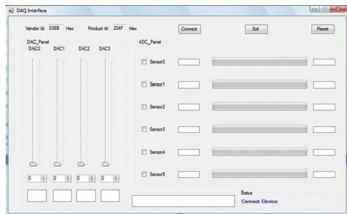


Fig. 7: GUI for DAQ device communication using C# program

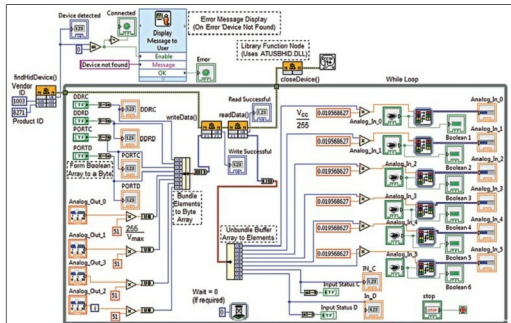


Fig. 8: LabView VI for the DAQ device interface

input pins by default.

The remaining four bytes are the values for DAC outputs. These are transferred to TLC7226's parallel input pins via port A of the microcontroller. To write a value to the DAC output, select the DAC output by A0 (pin 17) and A1 (pin 16) of the chip TLC7226, make the selected pin active by lowering WR pin (pin 15) of the chip, write the value to data pins of TLC7226 (DB0 through DB7) and finally make WR pin high before exiting the write subroutine.

The firmware sends in the first six bytes data collected from the analogue input pins of the microcontroller. The remaining two bytes sent by it contain the pin status of 8-bit wide port C and port D, respectively.

If you are not using the digital input pins, the firmware can be modified to read all the available eight analogue inputs of the microcontroller. By receiving and sending ten bytes, the code can be extended to use all the analogue inputs without disturbing the digital inputs. However, this reduces the sampling speed.

All the analogue inputs are sampled one by one and copied to the send buffer. It is advised to connect the unused analogue inputs to ground, to make the sampling stable. You are requested to go through the files board.c and GenericHID.c in LUFA101122\Demos\Device\LowLevel\GenericHID folder of the project for extensive reference. The codes are well remarked for future modifications. All the relevant source codes are included in this month's EFY DVD.

As the microcontroller has its own timers, the code can be extended to have a fixed sampling speed for any real-time application. Filters can also be implemented in the firmware itself for noiseless sensing.

The software imple-